

which carries out Java bytecode.) Because it's written in C, CPython has a Python interpreter and can be compiled for any CPU that C already has a compiler for.

The PyCon 2016 talk, “Playing with Python Bytecode” by Scott Sanderson and Joe Jevnik, is an excellent resource to learn more about this topic (<https://youtu.be/mxjv9KqzwjI>).

Script vs. Program, Scripting Language vs. Programming Language

The differences between a script and a program, or even a scripting language and a programming language, are vague and arbitrary. It's fair to say that all scripts are programs and all scripting languages are programming languages. But scripting languages are sometimes regarded as easier or “not real” programming languages.

One way to distinguish scripts from programs is by how the code executes. *Scripts* written in *scripting languages* are interpreted directly from the source code, whereas *programs* written in *programming languages* are compiled into binaries. But Python is commonly thought of as a scripting language, even though there is a compilation step to bytecode when a Python program is run. Meanwhile, Java isn't commonly thought of as a scripting language, even though it produces bytecode instead of machine code binaries, just like Python. Technically, *languages* aren't compiled or interpreted; rather, there are compiler or interpreter *implementations* of a language, and it's possible to create a compiler or interpreter for any language.

The differences can be argued but ultimately aren't very important. Scripting languages aren't necessarily less powerful, nor are compiled programming languages more difficult to work with.

Library vs. Framework vs. SDK vs. Engine vs. API

Using other people's code is a great time-saver. You can often find code to use packaged as libraries, frameworks, SDKs, engines, or APIs. The differences between these entities are subtle but important.